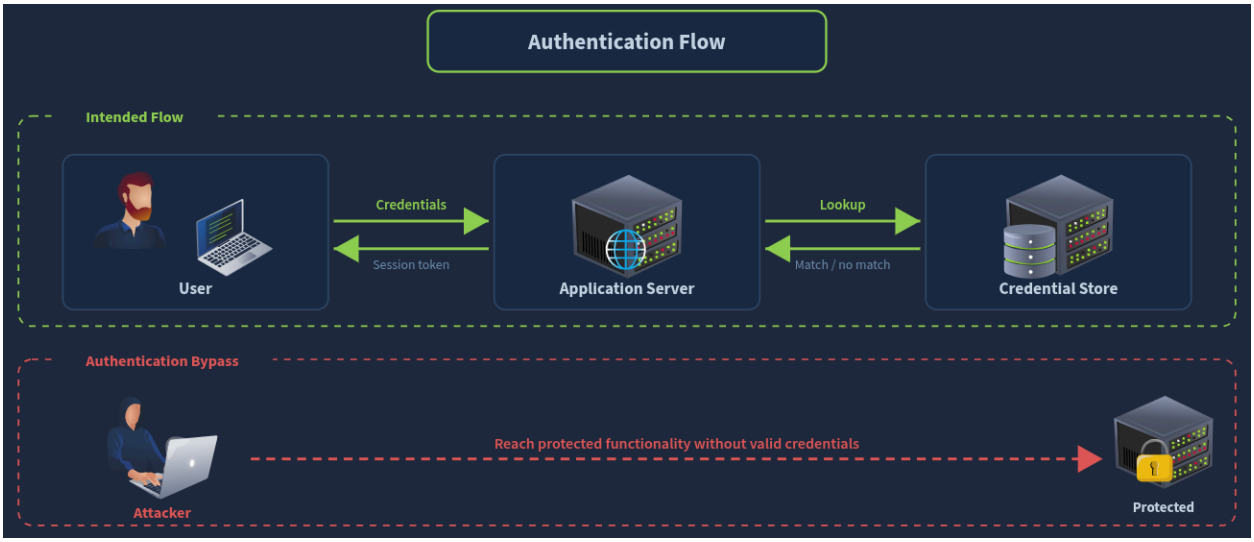
Jr.Pen 8.2. Broken Authentication

Task 1 - Introduction

Authentication verifies a user's identity against stored credentials and provides a session token to govern ongoing access.

An authentication bypass is an attack that grants unauthorized account access by exploiting flawed server assumptions or unverified data rather than stealing credentials.



Task 2 - Types of Authentication Bypass

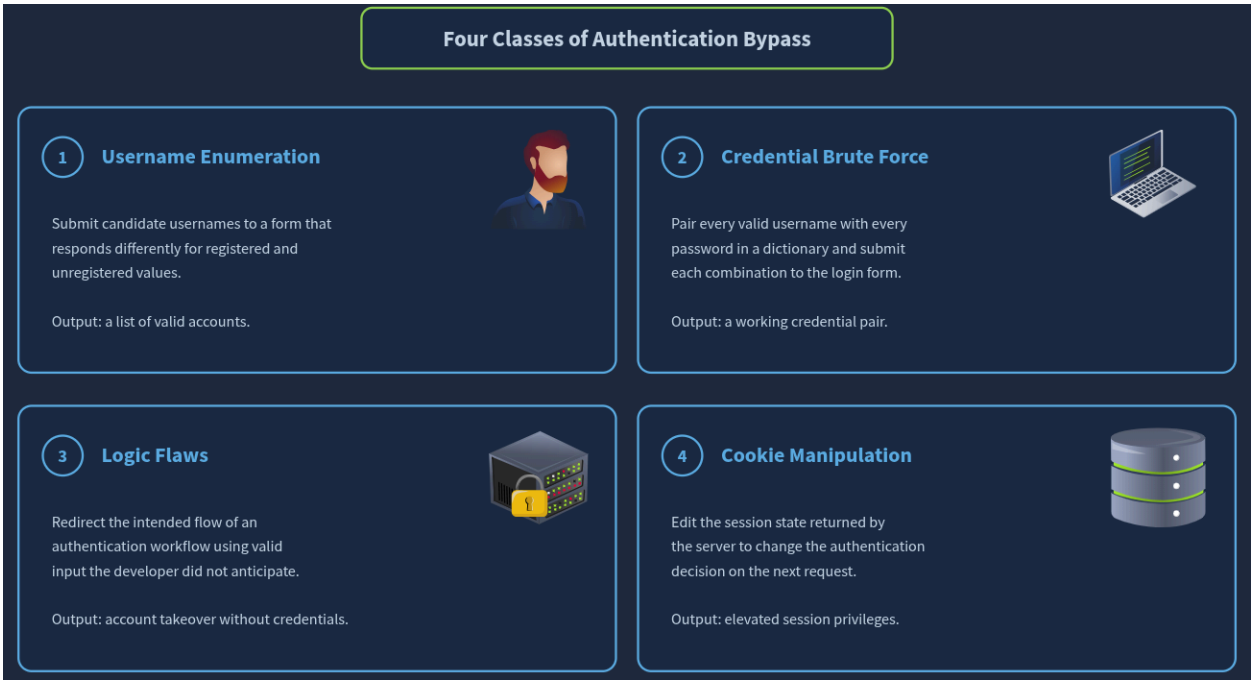
Four recurring flaw techniques target different authentication components: username enumeration, credential brute force, account recovery logic flaws, and cookie manipulation.

Username enumeration identifies valid application accounts by analyzing differences in server responses on forms like signup or password reset.

Credential brute force pairs valid usernames with common passwords to systematically breach accounts when rate limits are missing.

Logic flaws in multi-step recovery workflows allow attackers to manipulate parameters and redirect password reset links to foreign addresses.

Cookie manipulation alters unsigned session data on the client side to bypass server-side access controls on subsequent requests.



The impact of a successful authentication bypass depends directly on the compromised account's privilege level. Accessing a regular user account exposes personal data, while bypassing an administrator account grants full control over the application, database, and settings. Bypassing authentication often serves as an initial foothold to escalate privileges, access broader system data, or achieve remote code execution. Recovered credentials are frequently used in credential stuffing attacks across unrelated services due to widespread password reuse.

Answer the questions below

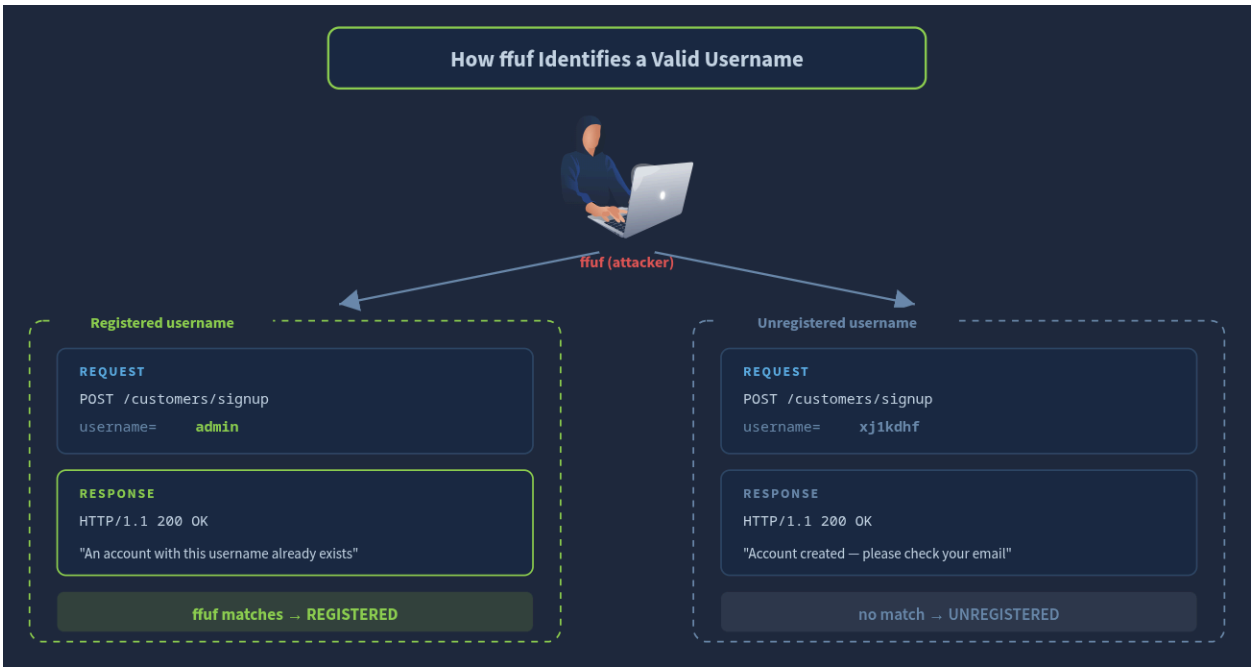
> What is the name for the practice of reusing credentials recovered from one application against unrelated applications?

Credential Reuse

Task 3 - Username Enumeration

Username enumeration is a preliminary reconnaissance technique that identifies valid accounts to feed into subsequent credential attacks. Applications leak account existence whenever forms process registered and unregistered values differently, such as on signup, login, or password reset pages.

Distinct error messages on registration forms reveal existing accounts by processing duplicate and new usernames differently. Applications returning identical error text can still be enumerated through subtle differences in response length, status codes, timing, or redirects.



Enumerating With ffuf - ffuf is a fast web fuzzer written in Go. It substitutes each entry from a wordlist into a marker inside an HTTP request and reports the responses that match a given condition.

The following command enumerates valid usernames against the Acme signup page:

```
ffuf -w /usr/share/wordlists/SecLists/Usernames/Names/names.t
xt -X POST -d "username=FUZZ&email=x&password=x&cpassword=x"
-H "Content-Type: application/x-www-form-urlencoded" -u htt
```

```
p://10.114.153.166/customers/signup -mr "username already exi
sts"
```

The -w argument selects the wordlist of candidate usernames. The -X POST argument sets the HTTP method, as required by the signup form. The -d argument defines the request body, with the token FUZZ acting as a placeholder that ffuf replaces with each word from the wordlist in turn. The -H argument sets the Content-Type header so the server treats the body as URL-encoded form data. The -u argument sets the target URL. Finally, the -mr argument (match regex) restricts the output to responses whose body contains the string username already exists.

Answer the questions below

> What is the username starting with si*** ?

simon

> What is the username starting with st*** ?

steve

> What is the username starting with ro**** ?

robert

run the ffuf code

```
root@ip-10-114-125-124:~# ffuf -w /usr/share/wordlists/SecLists/Usernames/Names/
names.txt -X POST -d "username=FUZZ&email=x&password=x&cpassword=x" -H "Content-
Type: application/x-www-form-urlencoded" -u http://10.114.153.166/customers/sign
up -mr "username already exists"

        /'___\  /'___\           /'___\
       /\ \__/ /\ \__/  __  __  /\ \__/
       \ \ ,__\\ \ ,__\/\ \/\ \ \ \ ,__\
        \ \ \_/ \ \ \_/\ \ \_\ \ \ \ \_/
         \ \_\   \ \_\  \ \____/  \ \_\
          \/_/    \/_/   \/___/    \/_/

       v2.1.0
________________________________________________

 :: Method           : POST
 :: URL              : http://10.114.153.166/customers/signup
 :: Wordlist         : FUZZ: /usr/share/wordlists/SecLists/Usernames/Names/names
.txt
 :: Header           : Content-Type: application/x-www-form-urlencoded
 :: Data             : username=FUZZ&email=x&password=x&cpassword=x
 :: Follow redirects : false
 :: Calibration      : false
 :: Timeout          : 10
```

Task 4 - Brute Forcing a Login Form

Credential brute-forcing systematically submits username-password combinations until finding a valid pair, depending heavily on wordlist size and application defenses. Brute force is only practical with small account lists, making preliminary username enumeration essential to avoid rate limits and lockouts.



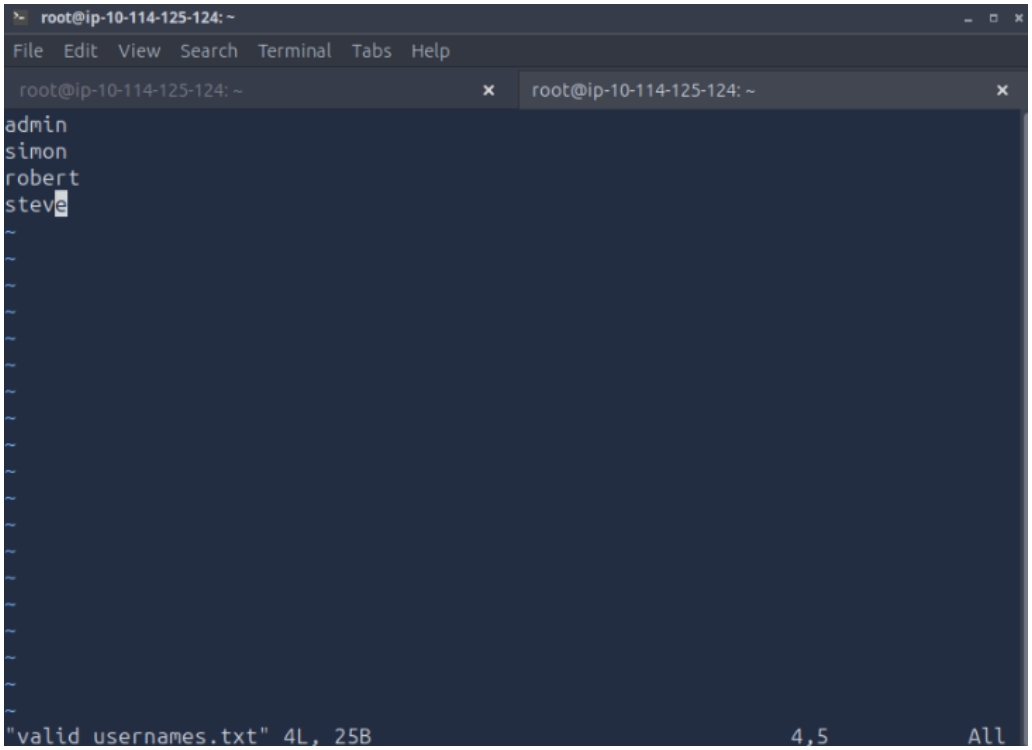
very brute-force tool needs a way to distinguish a successful login from a failed one. The Acme IT Support login form at http://10.114.153.166/customers/login returns HTTP 200 with the login page re-rendered when the submitted credentials are invalid, and a 302 redirect to the customer dashboard when the credentials are valid. The change in status code is the signal that a login has succeeded.

Answer the questions below

> What is the valid username and password (format: username/password)?

steve/thunder

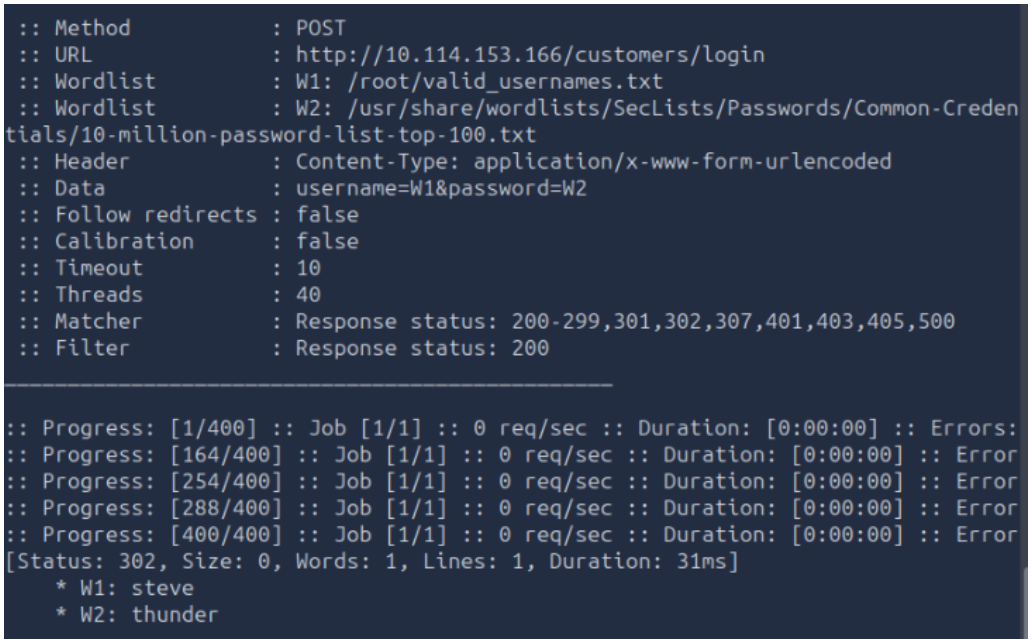
Create valid_usernames.txt



Run query:

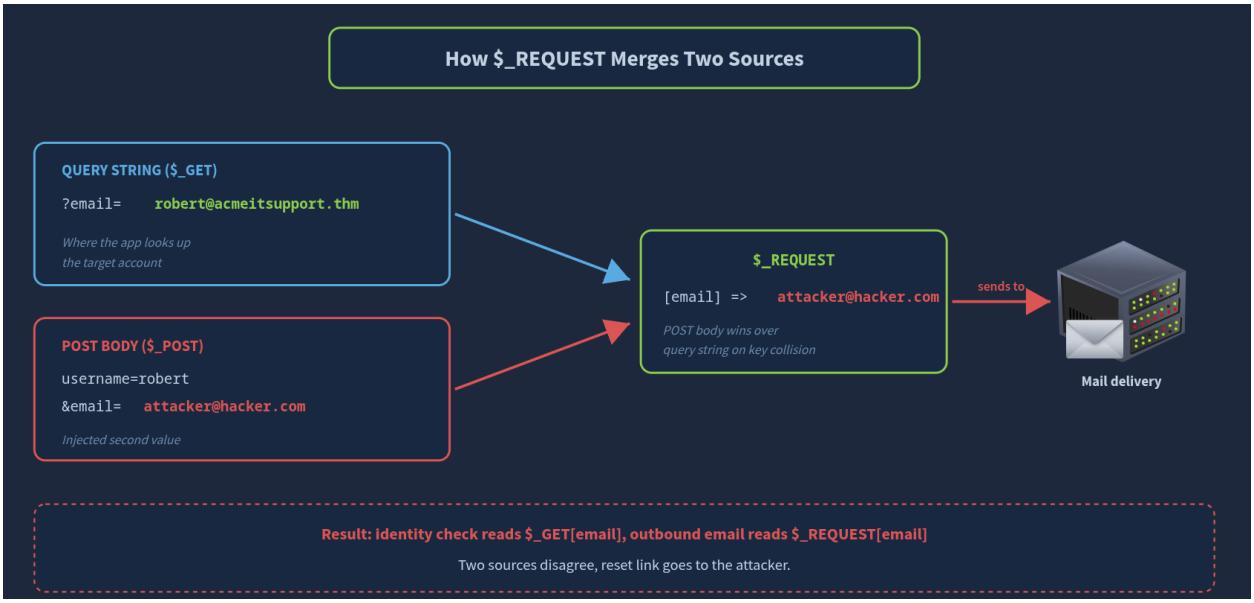
```
ffuf -w valid_usernames.txt:W1,/usr/share/wordlists/SecLists/
Passwords/Common-Credentials/10-million-password-list-top-10
0.txt:W2 -X POST -d "username=W1&password=W2" -H "Content-Typ
```

```
e: application/x-www-form-urlencoded" -u http://10.114.153.16
6/customers/login -fc 200
```

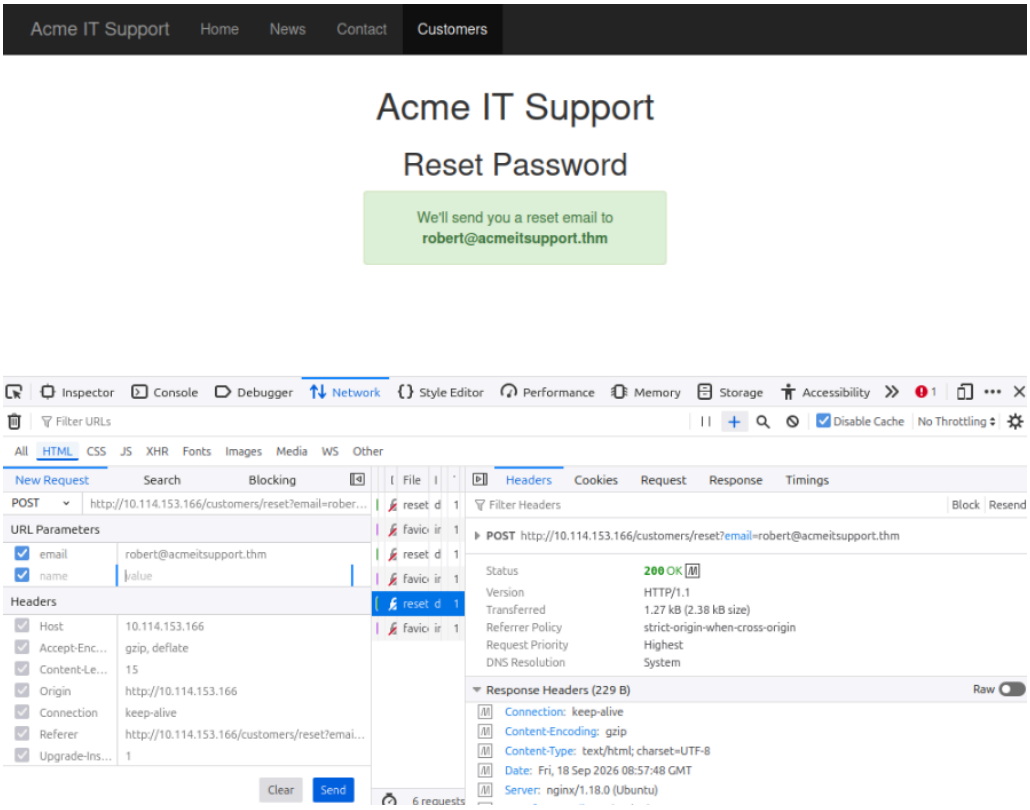


Task 5 - Logic Flaws

Logic flaws exploit inconsistencies between valid business rules to divert application workflows without relying on malformed data. Automated tools struggle to detect logic flaws because identifying them requires manually reasoning about an application's specific business context or code.



Submitting robert@acmeitsupport.thm and the username robert produces the confirmation shown below.



Answer the questions below

> What is the flag from Robert's support ticket?

THM{AUTH_BYPASS_COMPLETE}

Task 6 - Cookie Manipulation

Stateless HTTP applications rely on cookies sent with each request to maintain user authentication states without re-entering credentials. Modifying unsigned session cookies allows clients to alter the server's authentication decision and hijack arbitrary user accounts.

Hashed Cookies: Developers incorrectly rely on one-way hash functions in cookies to prevent tampering, misinterpreting uninvertibility for signature security. Hashes do not prevent cookie tampering because anyone who knows the underlying input can independently generate and substitute a matching hash value.

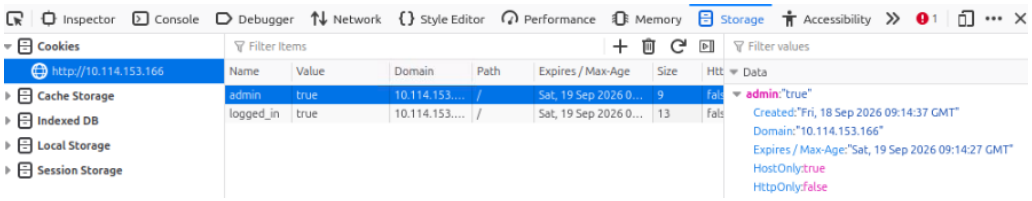
Encoded Cookies: Encoding reversibly translates structured data for protocol transport without providing any confidentiality or integrity protection against cookie tampering.

Answer the questions below

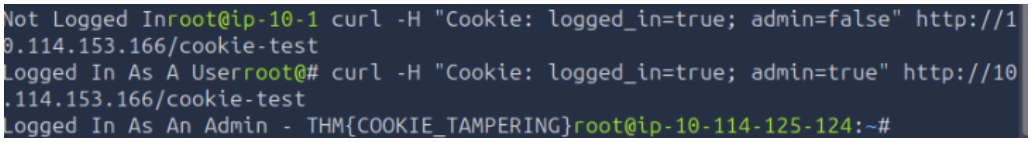
> What is the flag from changing the plain text cookie values?

THM{COOKIE_TAMPERING}

Add cookie

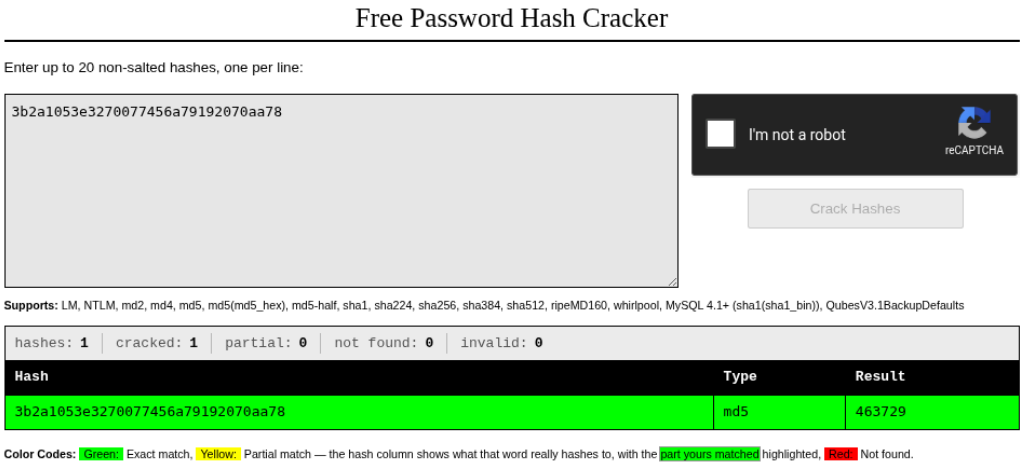


or run curl command



> What is the value of the md5 hash 3b2a1053e3270077456a79192070aa78 ?

463729



> What is the base64 decoded value of VEhNe0JBU0U2NF9FTkNPREIOR30= ?

THM{BASE64_ENCODING}

> Encode the following value using base64 {"id":1,"admin":true}

eyJpZCI6MSwiYWRtaW4iOnRydWV9

Task 7 - Conclusion

Authentication bypass vulnerabilities are usually caused by poorly coded web applications or poorly configured session management. There are, however, secure coding methods and design patterns that reduce the chances of being vulnerable to each class of attack.

Mitigating Username Enumeration: Username enumeration is mitigated by returning completely identical HTTP responses for all account states alongside implementing rate limits and CAPTCHAs to raise automated attack costs.

Mitigating Brute Force: Credential brute force is mitigated through rate limiting, account lockouts, multi-factor authentication, and password policies that prioritize long passphrases over short, complex ones.

Mitigating Logic Flaws: Logic flaws are mitigated by ensuring security workflows derive decisions strictly from a single, trusted server-side source rather than

merged request inputs.

Mitigating Cookie Tampering: Cookie tampering is prevented either by cryptographically signing session tokens with a server-side secret or by issuing opaque session IDs backed by server-side state.